รายงานสรุปผลการดำเนินงานรายบุคคล (Individual Report)

รหัส 008 — นายปภังกร ทองเจริญ

วิชา: DevSecOps | **โครงงาน:** Student Portfolio & Skill Passport | **กลุ่มที่:** 3 | มหาวิทยาลัยสวนดุสิต

1. ข้อมูลส่วนบุคคล และ 2. บทบาทหน้าที่ในโครงการ

ชื่อ-นามสกุล: นายปภังกร ทองเจริญ

รหัสนักศึกษา: 6511011860008 (008)

บทบาทหลัก: DevSecOps Pipeline & API Security Specialist

กลุ่มโครงงาน: กลุ่มที่ 3 (Student Portfolio & Skill Passport)

หน้าที่ความรับผิดชอบ: วางระบบและบริหารจัดการ Automated CI/CD Security Pipeline ผ่าน GitHub Actions, ติดตั้งและคอนฟิก Semgrep SAST, ตรวจสอบช่องโหว่ Dependencies ผ่าน npm audit, และดูแลความปลอดภัยของ API Proxy เชื่อมต่อบริการภายนอก (GitHub REST API)

3. รายการงานที่ดำเนินการด้วยตนเอง

- ✓ สร้างและกำหนดค่าไฟล์ GitHub Actions Workflow .github/workflows/sast-security.yml ครอบคลุม 6 ขั้นตอน
- ✓ ตั้งค่าเงื่อนไข Security Quality Gate ตรวจสอบความปลอดภัยโค้ดในทุก Pull Request และ Push สู่ main
- ✓ ติดตั้งและคอนฟิก Semgrep SAST Rulesets (p/security-audit, p/secrets, p/owasp-top-ten, p/typescript)
- ✓ ดำเนินการตรวจสอบช่องโหว่ Third-party Dependencies ด้วย npm audit และรัน npm audit fix
- ✓ พัฒนา API Service ใน src/app/api/github/route.ts พร้อมใส่ Strict Regular Expression Whitelist ป้องกัน SSRF
- ✓ ปรับปรุงตรรกะการตรวจสอบรหัสผ่านและการบันทึก AuditLog ใน src/app/api/auth/[...nextauth]/route.ts

4. หลักฐานการทำงาน (Commits & Artifacts)

- **Git Commits:** ace7cad (feat SAST pipeline & week 12 report), b01856a (feat security remediations & evidence)
- **เอกสารหลักฐาน:** .github/workflows/sast-security.yml, evidence/sast/semgrep-sast-report.md, evidence/sast/npm-audit-report.txt, evidence/before-after/fix3-authentication-and-input-validation.md

5. ฟังก์ชันหรือโค้ดสำคัญที่ตนเองพัฒนา

```
// ตัดตอนจาก src/app/api/github/route.ts (Strict Regex Whitelist ป้องกัน SSRF)
const GITHUB_USERNAME_REGEX = /^[a-zA-Z0-9-]{1,39}$/;

export async function GET(request: Request) {
  const { searchParams } = new URL(request.url);
  const username = searchParams.get('username');
  // ตัดการทำงานทันทีหากพบอักขระพิเศษ ป้องกัน SSRF / Path Traversal
  if (!username || !GITHUB_USERNAME_REGEX.test(username)) {
    return NextResponse.json({ error: 'Invalid username format' }, { status: 400 });
  }
  const res = await fetch(`https://api.github.com/users/${username}/repos`, {
    headers: { 'User-Agent': 'SDU-SkillPassport-App' }
  }); ...
}
```

6. ช่องโหว่ที่ตรวจพบ และ 7. วิธีแก้ไขและผลหลังแก้ไข

- **ช่องโหว่ที่ตรวจพบ:** OWASP A06: Vulnerable Components (10 ช่องโหว่ใน npm audit), OWASP A07: Weak Auth (ยอมรับรหัสผ่านว่าง), OWASP A10: Potential SSRF บน GitHub API Proxy
- **วิธีแก้ไข:** รัน npm audit fix, เพิ่มตรรกะตรวจสอบความยาวรหัสผ่านใน NextAuth พร้อมบันทึก AuditLog, เพิ่ม Strict Regex Whitelist คัดกรองชื่อผู้ใช้ก่อนส่งต่อภายนอก
- **ผลหลังแก้ไข:** Semgrep และ npm audit ผ่านการทดสอบระดับ High, ป้องกันการบายพาสรหัสผ่านสำเร็จ และการป้อนอักขระแปลกปลอมเข้า GitHub Proxy ถูกตัดทิ้งด้วย HTTP 400

8. ปัญหาที่พบและวิธีแก้ไข

ปัญหา: Semgrep บน GitHub Actions แจ้งเตือน False Positive กับไฟล์ Build Output (`.next/`) และ `node_modules/` ส่งผลให้ Pipeline ช้าและไม่ผ่าน
วิธีแก้: ปรับแต่งแคช cache: 'npm' และกำหนดขอบเขตการสแกนให้เจาะจงเฉพาะโฟลเดอร์ src/ และเลือกใช้ Rulesets ที่ตรงกับเทคโนโลยี

9. สิ่งที่ได้เรียนรู้ด้าน DevSecOps

หัวใจสำคัญคือการทำ **Shift-Left Security** ดึงการตรวจสอบความปลอดภัยมาไว้ในกระบวนการเขียนโค้ด ทำให้ตรวจพบช่องโหว่ได้ตั้งแต่เปิด Pull Request ก่อนที่จะส่งโค้ดขึ้นสู่ Production จริง

10. การประเมินตนเอง: 9.5 / 10 คะแนน

วางระบบ CI/CD Security Pipeline ผ่าน GitHub Actions ได้จริง พร้อมติดตั้ง SAST, SCA และแก้ไขช่องโหว่ด้าน Authentication และ SSRF ได้อย่างมีประสิทธิภาพ

11. การประเมินเพื่อนร่วมกลุ่ม

- **010 นายอภิสิทธิ์ (10/10):** ออกแบบฐานข้อมูลแข็งแกร่ง ป้องกัน SQLi และคุ้มครอง PDPA ได้อย่างรัดกุมมาก

• **009 นายปวีณวัชร์ (10/10):** พัฒนา UI สวยงาม ทันสมัย และคอนฟิก HTTP Security Headers และ Deploy Vercel ได้สมบูรณ์